

SQL HANDBOOK

PART - 1

Page - 1/8

1. INTRODUCTION TO SQL

• What is SQL ?

→ SQL (Structured Query Language) is a standard programming language used to create, modify, query and manage relational databases.

• Why SQL ?

→ SQL is used to store data, retrieve data, update data and perform various operations on relational databases.

Key Benefits

- Easy to learn
- Data integrity
- Data security
- Reduced redundancy
- Standard language
- Easy data retrieval

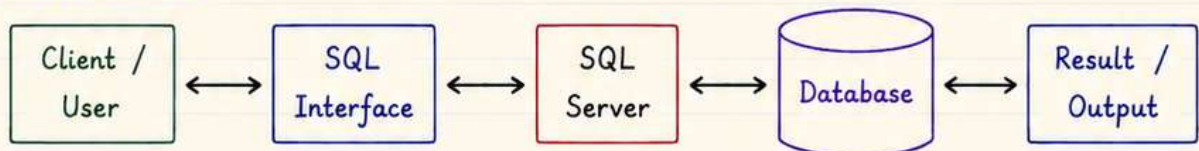
• Database, DBMS and RDBMS

→ **Database** : Collection of related data.

→ **DBMS** : Software used to manage the database.

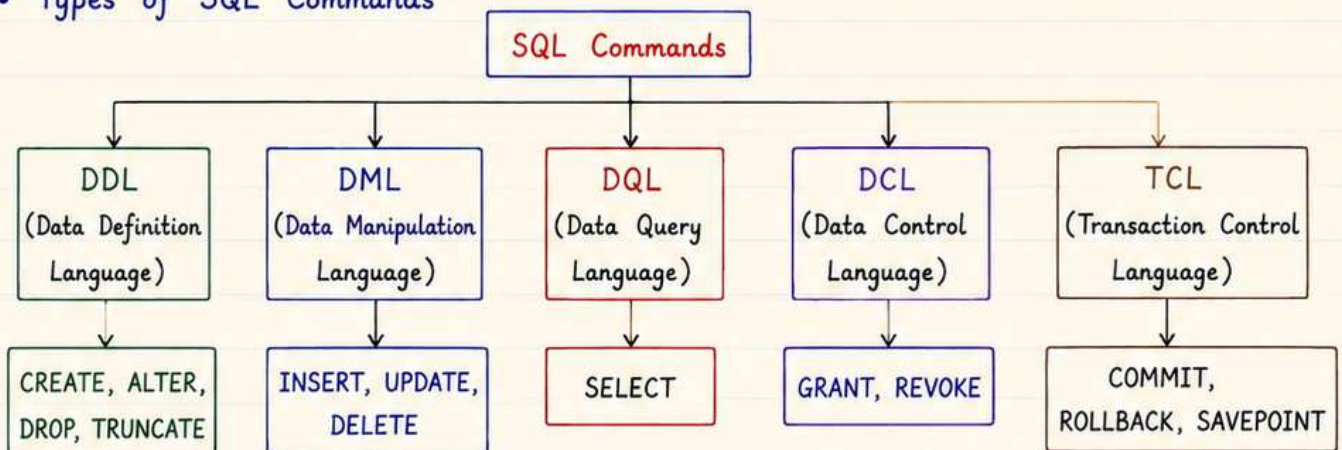
→ **RDBMS** : Relational Database Management System.

• SQL Architecture



Note : User sends a request → SQL processes it → Server interacts with database → Result is returned to the user.

• Types of SQL Commands



2. DATABASE, TABLES & DATA TYPES

- Create Database

```
CREATE DATABASE CompanyDB;
```

- Create Table

Column Name	Data Type	Description
EmpID	INT	Employee ID (Primary Key)
Name	VARCHAR(50)	Employee Name
Dept	VARCHAR(30)	Department Name
Salary	DECIMAL(10,2)	Employee Salary
JoinDate	DATE	Date of Joining
IsActive	BOOLEAN	Employee Active Status

SQL Query :

```
CREATE TABLE Employee (
    EmpID INT PRIMARY KEY,
    Name VARCHAR(50) NOT NULL,
    Dept VARCHAR(30),
    Salary DECIMAL(10,2),
    JoinDate DATE,
    IsActive BOOLEAN
);
```

- Common Data Types

INT — Integer numbers
VARCHAR(n) — String of maximum length n
DECIMAL(m,d) — Decimal numbers with m total digits and d digits after decimal
DATE — Date values
BOOLEAN — True / False values

Primary Key

A column (or combination of columns) that uniquely identifies each row in a table.

- Example Table : Employee

EmpID	Name	Dept	Salary	JoinDate	IsActive
1	Aman	IT	55000.00	2024-01-15	TRUE
2	Riya	HR	45000.00	2024-02-10	TRUE
3	Karan	Finance	50000.00	2024-03-12	TRUE

3. INSERT, UPDATE & DELETE

• INSERT

```
INSERT INTO TableName (Column1, Column2, ...)
VALUES (Value1, Value2, ...);
```

Example :

```
INSERT INTO Employee (EmpID, Name, Dept, Salary, JoinDate, IsActive)
VALUES (4, 'Neha', 'IT', 60000, '2024-04-18', TRUE);
```

Employee Table (Before Insert)

EmpID	Name	Dept	Salary	JoinDate	IsActive
1	Aman	IT	55000	2024-01-15	TRUE
2	Riya	HR	45000	2024-02-10	TRUE
3	Karan	Finance	50000	2024-03-12	TRUE



After INSERT (EmpID = 4)

EmpID	Name	Dept	Salary	JoinDate	IsActive
1	Aman	IT	55000	2024-01-15	TRUE
2	Riya	HR	45000	2024-02-10	TRUE
3	Karan	Finance	50000	2024-03-12	TRUE
4	Neha	IT	60000	2024-04-18	TRUE

• UPDATE

```
UPDATE TableName
SET Column1 = Value1, Column2 = Value2, ...
WHERE Condition;
```

Example :

```
UPDATE Employee
SET Salary = 60000
WHERE EmpID = 2;
```

Before UPDATE (EmpID = 2)

EmpID	Name	Dept	Salary	JoinDate	IsActive
1	Aman	IT	55000	2024-01-15	TRUE
2	Riya	HR	45000	2024-02-10	TRUE
3	Karan	Finance	50000	2024-03-12	TRUE
4	Neha	IT	60000	2024-04-18	TRUE



After UPDATE (EmpID = 2)

EmpID	Name	Dept	Salary	JoinDate	IsActive
1	Aman	IT	55000	2024-01-15	TRUE
2	Riya	HR	60000	2024-02-10	TRUE
3	Karan	Finance	50000	2024-03-12	TRUE
4	Neha	IT	60000	2024-04-18	TRUE

• DELETE

```
DELETE FROM TableName
WHERE Condition;
```

Example :

```
DELETE FROM Employee
WHERE EmpID = 3;
```

Before DELETE (EmpID = 3)

EmpID	Name	Dept	Salary	JoinDate	IsActive
1	Aman	IT	55000	2024-01-15	TRUE
2	Riya	HR	60000	2024-02-10	TRUE
3	Karan	Finance	50000	2024-03-12	TRUE
4	Neha	IT	60000	2024-04-18	TRUE



After DELETE (EmpID = 3)

EmpID	Name	Dept	Salary	JoinDate	IsActive
1	Aman	IT	55000	2024-01-15	TRUE
2	Riya	HR	60000	2024-02-10	TRUE
4	Neha	IT	60000	2024-04-18	TRUE

• TRUNCATE

```
TRUNCATE TABLE TableName;
```

→ Removes all rows from the table but keeps the table structure.

Example : TRUNCATE TABLE Employee;

After TRUNCATE

EmpID	Name	Dept	Salary	JoinDate	IsActive

Note :

DELETE removes selected rows.

TRUNCATE removes all rows.

4. SELECT & FILTERING

• SELECT

```
SELECT * FROM Employee; -- returns all columns
```

• WHERE

```
SELECT * FROM Employee WHERE Salary > 50000;
```

• DISTINCT

```
SELECT DISTINCT Dept FROM Employee;
```

• ORDER BY

```
SELECT * FROM Employee ORDER BY Salary ASC;
SELECT * FROM Employee ORDER BY Salary DESC;
```

• LIMIT / TOP

```
SELECT * FROM Employee LIMIT 3; -- MySQL
SELECT TOP 3 * FROM Employee; -- SQL Server
```

Sample Employee Table :

EmpID	Name	Dept	Salary	JoinDate
1	Aman	IT	55000	2024-01-15
2	Riya	HR	45000	2024-02-10
3	Karan	Finance	50000	2024-03-12
4	Neha	IT	60000	2024-04-18
5	Arjun	HR	35000	2024-05-20

Notes :

- WHERE is used to filter rows before any grouping or aggregation.
- ORDER BY is used to sort the result set.
- LIMIT / TOP is used to restrict number of rows returned.
- LIKE supports pattern matching.
 - % → any number of characters (including zero)
 - _ → single character

• BETWEEN

```
SELECT * FROM Employee
WHERE Salary BETWEEN 40000
AND 60000;
```

• IN

```
SELECT * FROM Employee
WHERE Dept IN ('IT', 'HR');
```

• LIKE

```
SELECT * FROM Employee
WHERE Name LIKE 'A%'; -- Starts with A
WHERE Name LIKE '%n'; -- Ends with n
WHERE Name LIKE '_a_'; -- a in middle
```

Sample Outputs :

1) WHERE Salary > 50000

EmpID	Name	Dept	Salary	JoinDate
1	Aman	IT	55000	2024-01-15
4	Neha	IT	60000	2024-04-18

2) DISTINCT Dept

Dept
IT
HR
Finance

3) ORDER BY Salary ASC

EmpID	Name	Dept	Salary	JoinDate
5	Arjun	HR	35000	2024-05-20
2	Riya	HR	45000	2024-02-10
3	Karan	Finance	50000	2024-03-12
1	Aman	IT	55000	2024-01-15
4	Neha	IT	60000	2024-04-18

5. SQL FUNCTIONS

- SQL Functions are predefined formulas that perform calculations and return a single value.

A. Aggregate Functions

COUNT()	- Counts rows
SUM()	- Returns total
AVG()	- Returns average
MIN()	- Returns minimum
MAX()	- Returns maximum

Example Table : Employee

EmpID	Name	Dept	Salary
1	Aman	IT	55000
2	Riya	HR	45000
3	Karan	Finance	50000
4	Neha	IT	60000
5	Arjun	HR	35000

Aggregate Function Examples :

- 1) COUNT() - counts total number of rows

```
SELECT COUNT(*) AS TotalEmployees
FROM Employee;
```

Output :

TotalEmployees
5

- 2) SUM() - returns total of salary

```
SELECT SUM(Salary) AS TotalSalary
FROM Employee;
```

Output :

TotalSalary
245000

- 3) AVG() - returns average salary

```
SELECT AVG(Salary) AS AverageSalary
FROM Employee;
```

Output :

AverageSalary
49000.00

- 4) MIN() - returns minimum salary

```
SELECT MIN(Salary) AS MinSalary
FROM Employee;
```

Output :

MinSalary
35000

- 5) MAX() - returns maximum salary

```
SELECT MAX(Salary) AS MaxSalary
FROM Employee;
```

Output :

MaxSalary
60000

B. Scalar / String Functions

UPPER()	- Converts to uppercase
LOWER()	- Converts to lowercase
LENGTH()	- Returns length of string
ROUND()	- Rounds a number
CONCAT()	- Joins two or more strings

Examples :

- 1) UPPER(Name)

```
SELECT UPPER(Name) AS NameUpper
FROM Employee;
```

Output :

NameUpper
AMAN
RIYA
KARAN
NEHA
ARJUN

- 2) LOWER(Name)

```
SELECT LOWER(Name) AS NameLower
FROM Employee;
```

Output :

NameLower
aman
riya
karan
neha
arjun

- 3) LENGTH(Name)

```
SELECT Name, LENGTH(Name) AS Len
FROM Employee;
```

Output :

Name	Len
Aman	4
Riya	4
Karan	5
Neha	4
Arjun	5

- 4) CONCAT(Name, '-', Dept)

```
SELECT CONCAT(Name, '-', Dept) AS Info
FROM Employee;
```

Output :

Info
Aman - IT
Riya - HR
Karan - Finance
Neha - IT
Arjun - HR

- 5) ROUND(AVG(Salary), 2)

```
SELECT ROUND(AVG(Salary), 2) AS AvgSal
FROM Employee;
```

Output :

AvgSal
49000.00

Note :

- Aggregate functions work on a set of rows and return a single value.
- Scalar functions work on individual values and return a single value.

6. GROUP BY & HAVING

- **GROUP BY** is used to group rows that have the same values in specified columns.
- **HAVING** is used to filter groups after grouping.

A. GROUP BY

```
SELECT Dept, COUNT(*) AS TotalEmployees
FROM Employee
GROUP BY Dept;
```

Sample Output :

Dept	TotalEmployees
IT	2
HR	2
Finance	1

B. HAVING

```
SELECT Dept, COUNT(*) AS TotalEmployees
FROM Employee
GROUP BY Dept
HAVING COUNT(*) > 1;
```

Sample Output :

Dept	TotalEmployees
IT	2
HR	2

C. Difference between WHERE and HAVING

WHERE	HAVING
→ Filters rows before grouping.	→ Filters groups after grouping.
→ Cannot be used with aggregate functions.	→ Can be used with aggregate functions.
→ Works on individual rows.	→ Works on grouped result.
→ Used with SELECT, UPDATE, DELETE.	→ Used only with GROUP BY.

Example Table : Employee

EmpID	Name	Dept	Salary
1	Aman	IT	55000
2	Riya	HR	45000
3	Karan	Finance	50000
4	Neha	IT	60000
5	Arjun	HR	35000

Note :

- **GROUP BY** comes after **WHERE** but before **HAVING**.
- **HAVING** works on groups, **WHERE** works on rows.

More Examples :

1) Average salary in each department

```
SELECT Dept, AVG(Salary) AS AvgSalary
FROM Employee
GROUP BY Dept;
```

Output :

Dept	AvgSalary
IT	57500.00
HR	40000.00
Finance	50000.00

2) Departments having more than 1 employee

```
SELECT Dept, COUNT(*) AS TotalEmployees
FROM Employee
GROUP BY Dept
HAVING COUNT(*) > 1;
```

Output :

Dept	TotalEmployees
IT	2
HR	2

3) Department with highest salary in each group

```
SELECT Dept, MAX(Salary) AS MaxSalary
FROM Employee
GROUP BY Dept;
```

Output :

Dept	MaxSalary
IT	60000
HR	45000
Finance	50000

7. SQL JOINS

- Joins are used to combine rows from two or more tables based on a related column.

Example Tables :

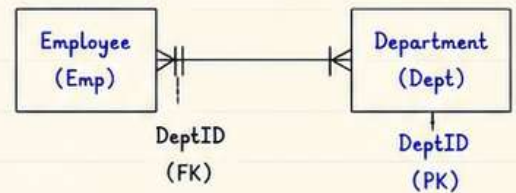
Employee Table (Emp)

EmpID	Name	DeptID
1	Aman	10
2	Riya	20
3	Karan	10
4	Neha	30
5	Arjun	NULL

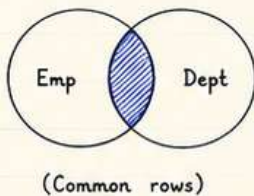
Department Table (Dept)

DeptID	DeptName
10	IT
20	HR
30	Finance
40	Marketing

Relationship Diagram :



1) INNER JOIN



- Returns only matching rows from both tables.

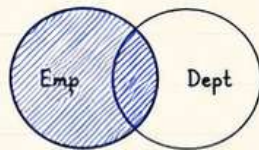
```

SELECT Emp.EmpID, Emp.Name, Dept.DeptName
FROM Employee Emp
INNER JOIN Department Dept
ON Emp.DeptID = Dept.DeptID;
  
```

Output :

EmpID	Name	DeptName
1	Aman	IT
2	Riya	HR
3	Karan	IT
4	Neha	Finance

2) LEFT JOIN



- Returns all rows from left table and matching rows from right.

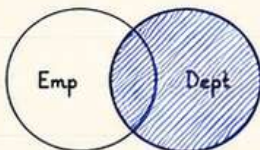
```

SELECT Emp.EmpID, Emp.Name, Dept.DeptName
FROM Employee Emp
LEFT JOIN Department Dept
ON Emp.DeptID = Dept.DeptID;
  
```

Output :

EmpID	Name	DeptName
1	Aman	IT
2	Riya	HR
3	Karan	IT
4	Neha	Finance
5	Arjun	NULL

3) RIGHT JOIN



- Returns all rows from right table and matching rows from left.

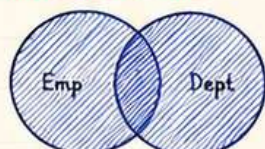
```

SELECT Emp.EmpID, Emp.Name, Dept.DeptName
FROM Employee Emp
RIGHT JOIN Department Dept
ON Emp.DeptID = Dept.DeptID;
  
```

Output :

EmpID	Name	DeptName
1	Aman	IT
3	Karan	IT
2	Riya	HR
4	Neha	Finance
NULL	NULL	Marketing

4) FULL OUTER JOIN



- Returns all rows when there is a match in either left or right table.

```

SELECT Emp.EmpID, Emp.Name, Dept.DeptName
FROM Employee Emp
FULL OUTER JOIN Department Dept
ON Emp.DeptID = Dept.DeptID;
  
```

Output :

EmpID	Name	DeptName
1	Aman	IT
2	Riya	HR
3	Karan	IT
4	Neha	Finance
5	Arjun	NULL
NULL	NULL	Marketing

Note :

- INNER JOIN returns only matched rows.
- LEFT/RIGHT/FULL JOIN may return NULL for unmatched rows.

8. SUBQUERIES & SET OPERATIONS

- Subqueries are queries nested inside another query.

A. SUBQUERIES

Types :

- Single Row Subquery (returns single value)
- Multiple Row Subquery (returns multiple values)
- Correlated Subquery (depends on outer query)

Examples :

1) Single Row Subquery

```
SELECT Name, Salary
FROM Employee
WHERE Salary = (SELECT MAX(Salary) FROM Employee);
```

→ Returns employee(s) having highest salary.

2) Multiple Row Subquery

```
SELECT Name, Dept
FROM Employee
WHERE Dept IN (SELECT Dept
FROM Department
WHERE DeptName = 'IT');
```

→ Returns employees who belong to department IT.

3) Correlated Subquery

```
SELECT E.Name, E.Salary
FROM Employee E
WHERE E.Salary > (SELECT AVG(Salary)
FROM Employee
WHERE Dept = E.Dept);
```

→ Compares salary of each employee with average salary of their department.

B. SET OPERATIONS

- Set operations are used to combine the result of two or more SELECT statements.

UNION

Returns all distinct rows from both queries.

```
SELECT Name FROM Employee
WHERE Dept = 'IT'
UNION
SELECT Name FROM Employee
WHERE Dept = 'HR';
```

Result :

Name
Aman
Riya
Karan
Neha

UNION ALL

Returns all rows from both queries including duplicates.

```
SELECT Name FROM Employee
WHERE Dept = 'IT'
UNION ALL
SELECT Name FROM Employee
WHERE Dept = 'HR';
```

Result :

Name
Aman
Riya
Karan
Riya
Neha
Arjun

INTERSECT

Returns only matching rows from both queries.

```
SELECT Name FROM Employee
WHERE Dept = 'IT'
INTERSECT
SELECT Name FROM Employee
WHERE Dept = 'HR';
```

Result :

Name
Riya
Neha

EXCEPT (or MINUS)

Returns rows from first query that are not in second query.

```
SELECT Name FROM Employee
WHERE Dept = 'IT'
EXCEPT
SELECT Name FROM Employee
WHERE Dept = 'HR';
```

Result :

Name
Aman
Karan

Notes :

- Number of columns and data types must be same in both queries.
- Order of columns must be the same.
- UNION removes duplicates, UNION ALL keeps duplicates.
- INTERSECT returns common rows, EXCEPT returns unmatched rows.

That's it for Part - 1

Happy Learning ! 😊

COMMENT 
SQL
♥ FOR ♥

SQL NOTES

FOLLOW
FOR PART 2 



TARGET 3K LIKES 

I WILL SHARE PART 2 ♥



SAVE



SHARE



FOLLOW



Aman Views

